STUDENT FIELD GUIDE

BATTLE BOTS

Program a robot in assembly language.
Then watch it fight.



⚙️ Instructions · 📡 Sensors · 🎯 Challenges · 🔑 Answer key

NAME _____ CLASS _____

01 Welcome, robot programmer!

You write the code. Your robot does the fighting.

In Battle Bots, you write a program that controls a robot tank. Then you press Start and watch it battle other robots. You can't steer with a keyboard or a controller. Once the match starts, **your code is in charge**.

You'll write your program in **assembly language**. It's one of the simplest computer languages there is. Each line is one tiny instruction, like "add 5" or "turn left" or "jump to another line." The chips inside phones, game consoles and cars run on instructions just like these.

🤖 **Fun fact**

Real assembly language is how programmers talk almost directly to a computer chip. You're learning the same ideas, just with a robot!

What's inside

01 **Welcome** — How a match works

02 **Big ideas** — Registers, labels, angles

03 **Every instruction** — What each command does

04 **Sensors** — How your robot "sees"

05 **Read a real robot** — Wanderer, line by line

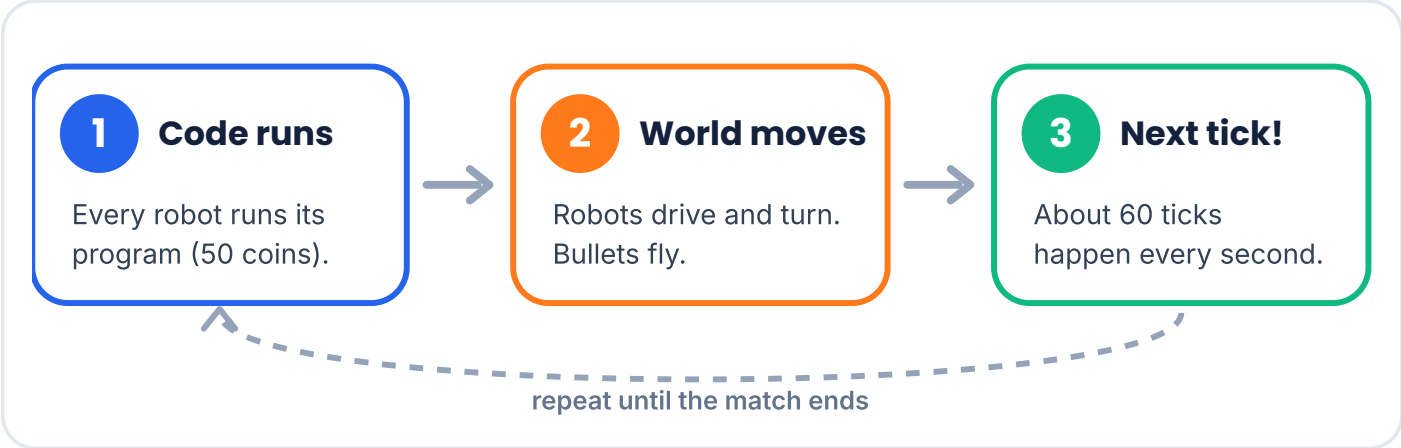
06 **Oops! Common mistakes** — And how to fix them

07 **Challenges** — Level up your robot

08 **Answer key** — Every challenge, solved

How a match works

The game runs in tiny steps called **ticks**. Every tick, three things happen:



Each robot gets **50 coins** (called cycles) to spend every tick. Most instructions cost 1 coin. `SCAN` and `RADAR` cost 3. Your program reads from top to bottom, one line at a time, unless a **jump** sends it somewhere else. When it reaches the bottom, it starts over at the top.

Winning a match

Arena rules

- Gray blocks are **obstacles**. They stop robots, bullets and scans.
- Brown patches are **mud**. In mud, robots can only go speed 2.
- Bumping a wall at full speed costs 2 health.

How to win

- Every robot starts with **100 health**.
- Each bullet hit takes away **10**.
- Last robot standing wins! If time runs out (6000 ticks), the most health wins.

Words to know

Instruction
One command, like `ADD` or `FIRE`.

Program
A list of instructions for your robot.

Tick
One tiny step of the game.

Register
A box that holds one number.

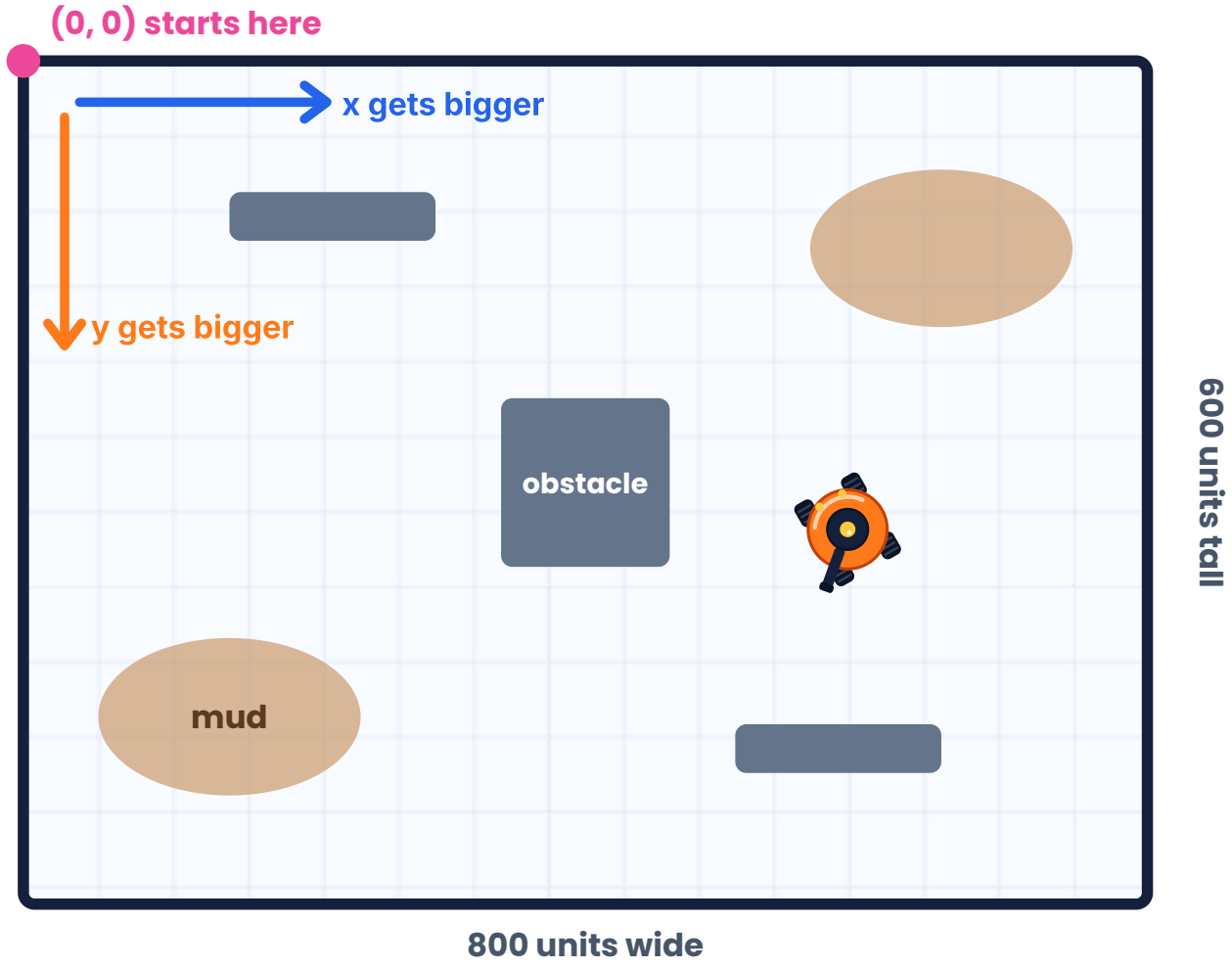
Label
A bookmark in your code.

Sensor
How your robot learns about the world.

🎮 **You're the coach, not the player**

Once you press Start, you can't help your robot. Everything it does comes from the code you wrote. Plan ahead!

The arena



A map of the arena. Notice that (0, 0) is the top-left corner!

Your first robot in 1 minute ⏱️

1 **Click "+ New robot"**
A code editor opens.

2 **Type the code**
Copy Spinner exactly.

3 **Press Apply**
(or Ctrl + Enter)

4 **Press ▶ Start**
Watch it go in circles!

spinner.asm

```
Spinner
main:
    SPEED 3      ; medium speed
    TURN 10      ; keep turning right
    WAIT
    JMP main
```

02 Big ideas to know first

Learn these and the rest gets easy.

Registers: your robot's scratch paper

A **register** is a small box that holds one number. Your robot has 8 of them, named `R0` through `R7`. They all start at 0. Think of them like the variables x and y in math class.



Eight boxes, eight numbers. Here R0 holds 10 and R2 holds 45.

Try it

```
MOV R0, 10     ; put 10 in box R0
ADD R0, 5      ; now R0 holds 15
```

Registers hold **whole numbers** only, so no decimals. Negative numbers are fine. If you ever need more than 8 boxes, your robot also has 256 **memory** slots, written `[0]` to `[255]`.

💬 Comments

Anything after a semicolon `;` is a **comment**. The computer skips it. Comments are notes for humans, so use them to explain your code!

```
SPEED 5    ; full speed ahead!
```

🔖 Labels

A **label** is a name with a colon, like `main:`. It's a bookmark in your code. Jumps use labels to know where to go.

```
main:
    SPEED 5
    JMP main
```

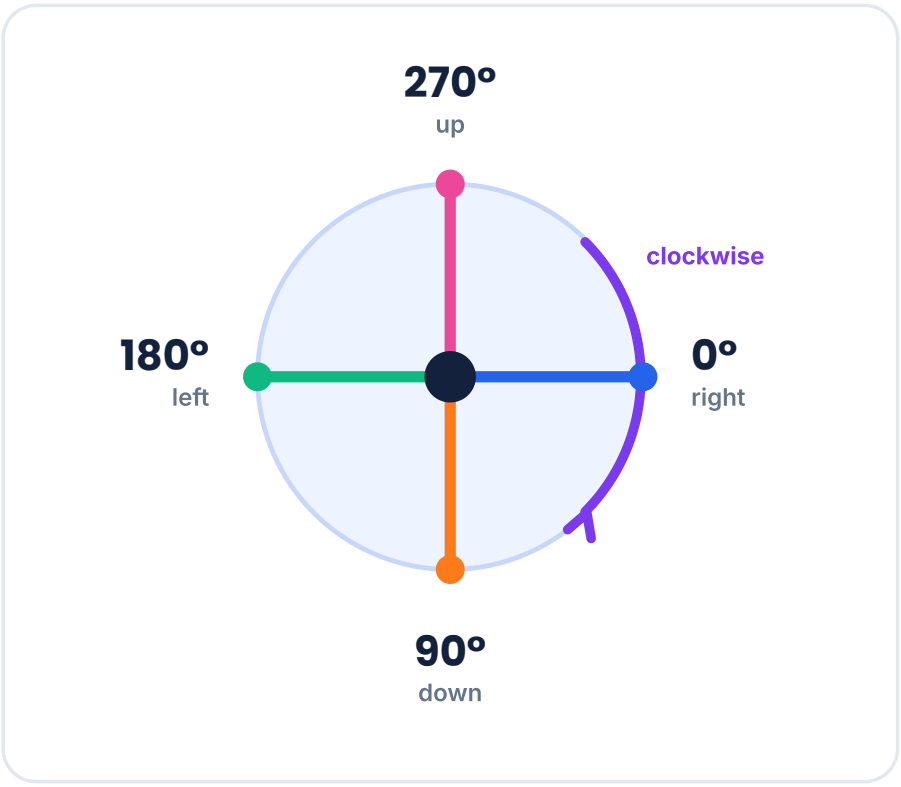
🏷️ Names for numbers

`.def` gives a number a name. It makes your code easier to read, and you only have to change the number in one place.

```
.def WALL_GAP 60
CMP R0, WALL_GAP   ; same as CMP R0, 60
```

Angles: which way is which?

Angles are in degrees, but they work a little differently than in math class. **0° points right**, and the numbers go **clockwise**, like a clock's hands. So 90° points **down**, not up!



Your robot has two parts that turn



🛞 The body

The body is what drives. It turns slowly, at most **8° per tick**.

🎯 The turret

The gun on top turns on its own, up to **20° per tick**. So your robot can drive one way and shoot another!

💡 **Orders, not magic**

When you say `SPEED 5`, your robot speeds up a little each tick, like a real car. It keeps following your last order until you give a new one.

03 Every instruction, explained

Your robot's whole vocabulary, in plain English.

Instructions follow a pattern: the **name**, then the things it works on, split by commas. When an instruction changes a value, the answer goes into the **first** thing listed.

```
ADD R0, 5     ; R0 = R0 + 5
;   ^   ^
;   |   +-- the amount to add
;   +------ where the answer goes
```

instruction · register · number · sensor · label · name for a number · comment

Moving numbers DATA

Instruction	What it does	Example	In plain English
`MOV a, b`	Copies b into a	`MOV R1, 90`	*Put 90 into R1.*
`PUSH a`	Puts a on top of the stack	`PUSH R3`	*Save R3 on the pile.*
`POP a`	Takes the top of the stack into a	`POP R3`	*Grab the top of the pile.*

The **stack** is like a pile of plates. You can only add or take from the top. It holds 64 numbers.

Math MATH

Instruction	What it does	Example	In plain English
`ADD a, b`	a = a + b	`ADD R0, 5`	*Add 5 to R0.*
`SUB a, b`	a = a − b	`SUB R0, R1`	*Take R1 away from R0.*
`MUL a, b`	a = a × b	`MUL R0, 3`	*Triple R0.*
`DIV a, b`	a = a ÷ b, no remainder	`DIV R0, 2`	*Cut in half. 7 ÷ 2 gives 3.*
`MOD a, b`	a = the remainder of a ÷ b	`MOD R0, 360`	*Keep R0 from 0 to 359.*
`INC a`	Add 1	`INC R2`	*Count up.*
`DEC a`	Subtract 1	`DEC R2`	*Count down.*
`NEG a`	Flip the sign	`NEG R1`	*5 becomes −5.*
`ABS a`	Make it positive	`ABS R1`	*−20 becomes 20.*
`MIN a, b`	Keep the smaller one	`MIN R0, 45`	*Never above 45.*
`MAX a, b`	Keep the bigger one	`MAX R0, -45`	*Never below −45.*
`RAND a, b`	Random number, 0 to b − 1	`RAND R0, 360`	*Pick any direction.*

⚠️ **Watch out!**
Dividing by zero (`DIV` or `MOD` by 0) crashes your robot's program.

Geometry helpers MATH

Instruction	What it does	Example
`NORM a`	Squeezes an angle into −180 to 179. Shows the shortest way to turn: negative = left, positive = right.	`NORM R0`
`ATAN2 a, dy, dx`	Finds the angle that points toward a spot.	`ATAN2 R0, R2, R1`
`SIN` / `COS`	Sine and cosine × 1000 (no decimals allowed!)	`SIN R1, R0`
`SQRT a, b`	Square root, rounded down	`SQRT R0, R0`

Bonus for curious coders: `AND`, `OR` and `XOR` work on the 1s and 0s (bits) inside a number. Most robots never need them.

Making decisions DECISIONS

This is the most important part of programming. It's how your robot makes choices! It always takes **two steps**:

Step 1: Compare

`CMP a, b` compares two numbers. It doesn't change anything. It just remembers which one is bigger.

Step 2: Jump (maybe)

A jump checks that answer. If it's true, it jumps to a label. If not, it goes to the next line.

Instruction	Jumps when…	Think of it as
`JMP label`	Always	*Go!*
`JE label`	a **equals** b	*If a = b, go.*
`JNE label`	a does **not equal** b	*If a ≠ b, go.*
`JG label`	a is **greater** than b	*If a > b, go.*
`JGE label`	a is **greater than or equal** to b	*If a ≥ b, go.*
`JL label`	a is **less** than b	*If a < b, go.*
`JLE label`	a is **less than or equal** to b	*If a ≤ b, go.*

It's an IF statement: if health is under 30, then run away!

```
1    GET  R0, HEALTH     ; R0 = my health
2    CMP  R0, 30         ; compare my health with 30
3    JL   run_away       ; if health < 30, jump to run_away
4    SPEED 5             ; otherwise, keep charging!
```

Subroutines and timing FLOW

Instruction	What it does
`CALL label`	Jumps to a label **and remembers where you came from**. Like pausing a video to do a chore, then coming back.
`RET`	Goes back to the line right after the `CALL`. Every `CALL` needs a `RET`.
`WAIT`	"I'm done for this tick." Ends your turn so the game can move everything.
`NOP`	Does nothing ("no operation"). Uses up one coin.
`HALT`	Stops your program for good. Your robot keeps following its last orders.

A chunk of code you `CALL` is called a **subroutine**. You write it once (like "dodge a bullet") and use it from anywhere.

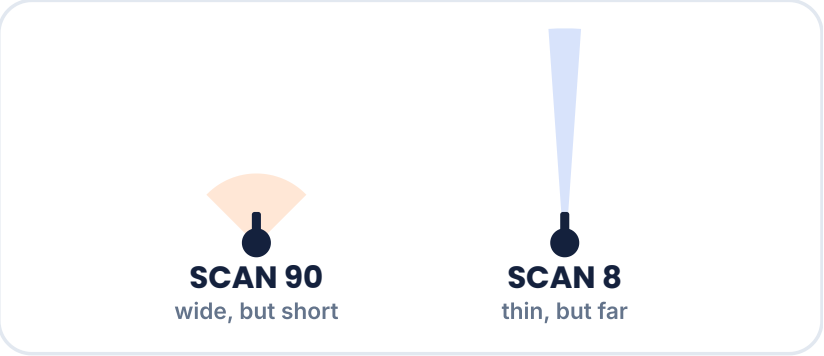
Controlling the robot ROBOT

Instruction	What it does	Example	In plain English
`GET a, SENSOR`	Reads a sensor into a register	`GET R0, HEALTH`	*How much health do I have?*
`SPEED n`	Speed from −3 (backward) to 5 (full speed)	`SPEED 5`	*Floor it!*
`TURN n`	Turn n degrees from where you face now. + = right, − = left	`TURN -90`	*Quarter turn left.*
`HEAD n`	Turn to face an exact direction	`HEAD 270`	*Face up.*
`AIM n`	Point the turret at an exact direction	`AIM 0`	*Point the gun right.*
`FIRE`	Shoot, if the gun is ready	`FIRE`	*Bang!*
`SCAN width`	Look for the nearest enemy in a cone around the turret	`SCAN 20`	*Look through binoculars.*
`RADAR`	Look for a bullet flying at you (within 250 units)	`RADAR`	*Is anyone shooting at me?*

After you `FIRE`, the gun needs **15 ticks** to reload. `FIRE` does nothing while it's reloading.

🔦 SCAN is like a flashlight

A wide beam lights up a big area but doesn't reach far. A narrow beam reaches really far, but only in a thin slice.



04 Sensors

Your robot's eyes and ears.

Read any sensor with `GET`. For example, `GET R0, FRONT` puts the distance to the wall ahead into R0.

About me

Sensor	What it tells you
`X`, `Y`	Where I am. X goes 0 → 800 (left to right). Y goes 0 → 600 (top to bottom).
`HEADING`	Which way my body is facing (0–359°)
`SPEED`	How fast I'm moving right now
`HEALTH`	How much health I have left (0–100)
`TURRET`	Which way my gun is pointing (0–359°)
`COOLDOWN`	Ticks until I can fire again. 0 means ready!
`FRONT`	How far I can drive straight ahead before I hit something
`MUD`	1 if I'm in mud, 0 if not
`LAST_HIT`	Ticks since a bullet last hit me (−1 if never)

What my SCAN found

Sensor	Tells you
`SCAN_DIST`	How far the enemy is. **−1 = nobody found**
`SCAN_ANGLE`	Which way the enemy is
`SCAN_X` `SCAN_Y`	Enemy's position
`SCAN_HEADING`	Where it's driving
`SCAN_SPEED`	How fast it's going
`SCAN_RANGE`	How far my scan reached

What my RADAR found

Sensor	Tells you
`THREAT_DIST`	How far the bullet is. **−1 = no danger**
`THREAT_ANGLE`	Which way the bullet is
`THREAT_HEADING`	Which way it's flying

About the match

Sensor	Tells you
`TICK`	The match clock
`ENEMIES`	Enemies still alive
`ALLIES`	Teammates alive
`ARENA_W` `ARENA_H`	800 and 600



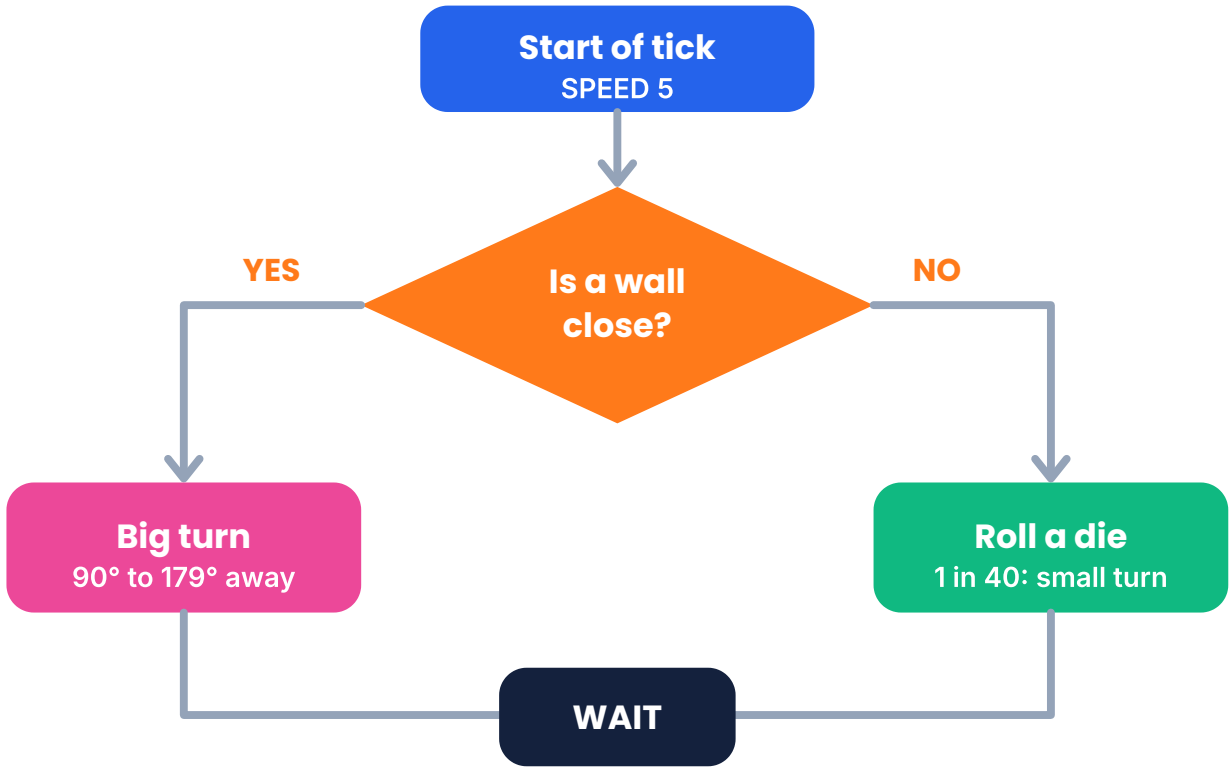
Always check for -1 first!

If `SCAN_DIST` or `THREAT_DIST` is −1, nothing was found, and the other values are just 0.

05 Let's read a real robot

Meet Wanderer. It just drives around the arena.

Wanderer turns away from walls, and sometimes it changes direction at random so it doesn't get boring. Before reading the code, look at this picture of what it "thinks" about every tick:



Wanderer's brain, one tick at a time

🧠 **The big idea**

Every tick, Wanderer asks one question: **Is a wall close?** If yes, it makes a big turn. If no, it rolls a 40-sided die. On a 0, it makes a small turn. That's the whole robot!

💡 **Try it yourself**

Type in Wanderer (next page), press Apply and Start, then press **Step** a few times. Watch the yellow marker move through the lines!

Line by line

The line numbers in the code match the numbers in the table.

wanderer.asm

```
1  Wanderer
2  .shape circle
3  .drive wheels
4  .turret short
5
6  .def WALL_GAP 60
7
8  main:
9      SPEED 5
10     GET  R0, FRONT
11     CMP  R0, WALL_GAP
12     JG   wander
13     RAND R1, 90
14     ADD  R1, 90
15     TURN R1
16     JMP  end
17 wander:
18     RAND R1, 40
19     CMP  R1, 0
20     JNE  end
21     RAND R1, 120
22     SUB  R1, 60
23     TURN R1
24 end:
25     WAIT
26     JMP  main
```

Line	What it means
1	The robot's name.
2–4	How the robot looks. Just for style!
6	"WALL_GAP" now means 60.
8	Bookmark: the loop starts here.
9	Drive at full speed.
10	How far until I hit a wall? Put it in R0.
11	Compare that distance with 60.
12	More than 60 away? All good! Jump to **wander**.
13	Wall is close! Pick a number from 0 to 89.
14	Add 90. Now R1 is 90 to 179.
15	Make that big turn, away from the wall.
16	Already turned, so skip to **end**.
17	Bookmark for the random part.
18–20	Roll a 40-sided die. Not a 0? Skip to **end**.
21–22	Rolled a 0! Pick a small turn, −60 to 59.
23	Make that small turn.
24	Bookmark: the end of the loop.
25	Done for this tick. Let the game move me.
26	Back to the top for the next tick!

06 Oops! Common mistakes

Every programmer makes these. Here's how to fix them.

1 Forgetting WAIT

Without `WAIT`, your loop runs until it uses up all 50 coins, and your robot acts weird. Put `WAIT` at the end of your main loop.

2 Jumping without CMP

Jumps only check the **last** `CMP`. Math like `ADD` doesn't count. Always compare right before you jump.

3 Using an empty scan

If `SCAN_DIST` is −1, nobody was found. Check for −1 first and skip the shooting part.

4 Mixing up angles

Remember: 0° is right, 90° is **down**, and angles go clockwise.

5 Expecting instant moves

Speed and turning change a little each tick. Give your robot time to get there.

6 CALL without RET

Every subroutine you `CALL` must end with `RET`, or your program gets lost.

7 Dividing by zero

`DIV` or `MOD` by 0 crashes your program. A blinking warning sign on your robot means it crashed.

8 Comment on the name line

Line 1 is only your robot's name. Put comments on line 2 or later.

Debugging like a pro 🔍

Slow it down

Press **Step** to move forward one tick at a time. The **yellow marker** shows the next line your robot will run.

Look inside

The **CPU inspector** shows every register and sensor. Is R0 what you expected?

💡 **Typos are easy to find**

If your code has a mistake, the editor shows an error. Click it to jump right to that line.

07 Challenges

Start from Wanderer and change it step by step. Test after every change!

CHALLENGE 1 ★☆☆☆☆

☐ Speed demon

Change `WALL_GAP` to 120. What happens? Now try 20.

CHALLENGE 2 ★☆☆☆☆

☐ Slow and steady

Make Wanderer drive at speed 2 instead of 5.

CHALLENGE 3 ★★☆☆☆

☐ Always turn right

Change the wall turn so it's always exactly 90°.

CHALLENGE 4 ★★★☆☆

☐ Reverse gear

When a wall is close, back up (`SPEED -3`) instead of turning.

CHALLENGE 5 ★★★☆☆

☐ Look around

Add `SCAN 30` and make the turret spin 15° every tick.

CHALLENGE 6 ★★★★☆

☐ First shot

If `SCAN_DIST` is not −1, `AIM` at `SCAN_ANGLE` and `FIRE` .

CHALLENGE 7 ★★★★☆

☐ Panic mode

If `HEALTH` drops below 30, go full speed and turn more often.

CHALLENGE 8 · BOSS LEVEL ★★★★★

☐ Bullet dodger

Use `RADAR` to dodge bullets. Look at the Dodger example robot for ideas!

How to test a challenge

1. **Copy Wanderer** and give it a new name on line 1.
2. **Change one thing** Just one! Small steps are easier to fix.
3. **Apply, then Start** Does it do what you wanted?
4. **Not working?** Use Step and the CPU inspector.

✅ **Check them off**

Tick the box on each card when your robot does it. Stuck? The answer key starts on the next page, but give it a real try first!

08 Answer key

Every challenge, solved. Green lines are new or changed.

💡 **How to read this**

Each answer shows what to change and why it works. Lines with a **green bar** are the new or changed lines. Every program here was tested in the real game.

1 Speed demon

Change one number

What to change

Change line 6 from `.def WALL_GAP 60` to 120, then try 20.

Why it works

With 120, Wanderer starts turning sooner, so it stays farther from walls. With 20, it waits until the last second, can't turn fast enough, and bumps walls a lot. Bumps cost health!

BEFORE

```
.def WALL_GAP 60
```

→ AFTER

```
.def WALL_GAP 120
```

2 Slow and steady

Change one number

What to change

Change `SPEED 5` to `SPEED 2`.

Why it works

A slower robot almost never crashes into walls, because it has time to turn. The downside? It's an easier target!

BEFORE

```
main:
    SPEED 5
```

→ AFTER

```
main:
    SPEED 2
```

3 Always turn right

Swap three lines for one

What to change

Replace the three lines that pick a random turn with `TURN 90`.

Why it works

Positive turns go right. The robot asks for a 90° right turn each tick until the wall is out of the way.

BEFORE

```
    RAND R1, 90
    ADD  R1, 90
    TURN R1
```

→ AFTER

```
    TURN 90
```

4 Reverse gear

Use a register as a countdown timer

What to change

When a wall is close, put 18 in `R2`. While R2 is bigger than 0, drive backward and count down by 1 each tick.

Why it works

If you only back up, you'll drive right back into the same wall! So the robot also turns while it reverses. When the countdown hits 0, it drives forward again.

reverser.asm

```
Reverser
.def WALL_GAP   60
.def BACK_TICKS 18  ; back-up time
main:
    CMP  R2, 0      ; counting down?
    JG   backing    ; yes: back up
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    MOV  R2, BACK_TICKS ; start!
    RAND R1, 90
    ADD  R1, 90
    TURN R1         ; turn as we go
    JMP  end
backing:
    SPEED -3        ; reverse!
    DEC  R2         ; count down
    JMP  end
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  end
    RAND R1, 120
    SUB  R1, 60
    TURN R1
end:
    WAIT
    JMP  main
```

5 Look around

Add a turret section

What to change

Add a `gun:` section before `end:`. Change the two `JMP end` / `JNE end` jumps to go to `gun` so the turret code runs every tick.

Why it works

Each tick, the robot reads where its turret points, adds 15°, and aims there. The turret can turn 20° per tick, so 15° is easy. Round and round it goes!

lookout.asm

```
Lookout
.def WALL_GAP 60
main:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  gun
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  gun
    RAND R1, 120
    SUB  R1, 60
    TURN R1
gun:
    SCAN 30         ; look around
    GET  R3, TURRET ; where's my gun?
    ADD  R3, 15     ; 15° more
    AIM  R3         ; spin!
end:
    WAIT
    JMP  main
```

6 First shot

Shoot when you see someone

What to change

After `SCAN 30`, check `SCAN_DIST`. If it's −1, keep spinning. Otherwise aim at `SCAN_ANGLE` and fire.

Why it works

`JL spin` jumps when SCAN_DIST is less than 0, which only happens when nobody was found. If someone *was* found, we skip right past it to the aiming code.

sharpshooter.asm

```
Sharpshooter
.def WALL_GAP 60
main:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  gun
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  gun
    RAND R1, 120
    SUB  R1, 60
    TURN R1
gun:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   spin        ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3          ; aim at them
    FIRE             ; bang!
    JMP  end
spin:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3          ; keep spinning
end:
    WAIT
    JMP  main
```

7 Panic mode

Use registers to hold settings

What to change

Keep the speed in `R4` and the dice size in `R5`. Normally use speed 3 and a 40-sided die. If health is under 30, switch to speed 5 and a 10-sided die.

Why it works

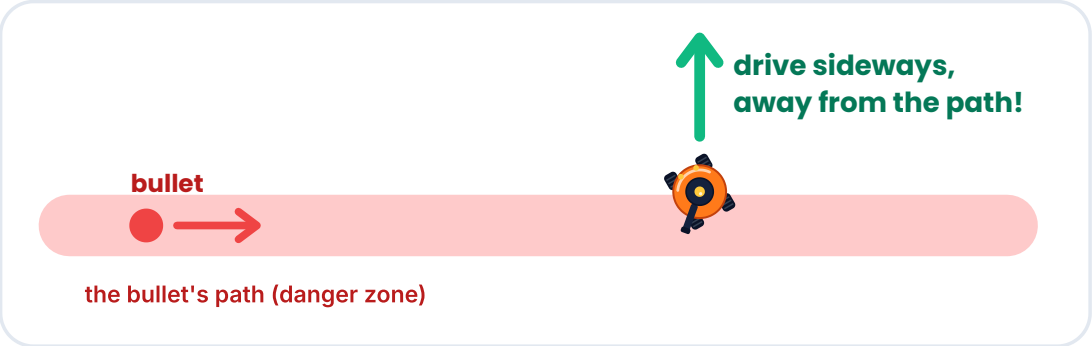
A 10-sided die lands on 0 much more often than a 40-sided one, so the robot turns 4 times as often. `SPEED R4` and `RAND R1, R5` use whatever is in the boxes!

panicker.asm

```
Panicker
.def WALL_GAP 60
main:
    MOV  R4, 3       ; normal speed
    MOV  R5, 40      ; 40-sided die
    GET  R0, HEALTH
    CMP  R0, 30
    JGE  calm        ; 30+? stay calm
    MOV  R4, 5       ; PANIC! go fast
    MOV  R5, 10      ; 10-sided die
calm:
    SPEED R4
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  gun
wander:
    RAND R1, R5      ; roll the die
    CMP  R1, 0
    JNE  gun
    RAND R1, 120
    SUB  R1, 60
    TURN R1
gun:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   spin        ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3          ; aim at them
    FIRE             ; bang!
    JMP  end
spin:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3          ; keep spinning
end:
    WAIT
    JMP  main
```

8 Bullet dodger

Boss level: RADAR plus a subroutine



boss.asm

```
Boss
.def WALL_GAP 60
main:
    RADAR              ; any bullets coming?
    GET  R0, THREAT_DIST
    CMP  R0, 0
    JL   roam          ; -1 means no danger
    CALL dodge         ; danger! go dodge
    JMP  gun
roam:
    SPEED 5
    GET  R0, FRONT
    CMP  R0, WALL_GAP
    JG   wander
    RAND R1, 90
    ADD  R1, 90
    TURN R1
    JMP  gun
wander:
    RAND R1, 40
    CMP  R1, 0
    JNE  gun
    RAND R1, 120
    SUB  R1, 60
    TURN R1
gun:
    SCAN 30
    GET  R3, SCAN_DIST
    CMP  R3, 0
    JL   spin          ; -1 = nobody
    GET  R3, SCAN_ANGLE ; which way?
    AIM  R3            ; aim at them
    FIRE               ; bang!
    JMP  end
spin:
    GET  R3, TURRET
    ADD  R3, 15
    AIM  R3            ; keep spinning
end:
    WAIT
    JMP  main
```

What to change

Start each tick with `RADAR`. If a bullet is coming, `CALL dodge`. If not, wander like normal. Either way, run the gun code.

Why it works

The dodge subroutine works out which side of the bullet's path the robot is on. Then it drives sideways, even farther to that side, so the bullet misses.

boss.asm (continued, at the very bottom)

```
dodge:
    GET  R1, THREAT_ANGLE
    ADD  R1, 180       ; from bullet to me
    GET  R2, THREAT_HEADING
    SUB  R1, R2        ; left or right of its path?
    NORM R1
    CMP  R1, 0
    JL   dodge_left
    ADD  R2, 90        ; on the right: go right
    JMP  dodge_go
dodge_left:
    SUB  R2, 90        ; on the left: go left
dodge_go:
    HEAD R2            ; face sideways
    SPEED 5            ; and go!
    RET                ; back to the CALL
```

You did it!

Your robot can roam, scan, shoot, panic and dodge. Now invent your own strategy and battle your friends!